The absolute minimum Git tutorial

a.k.a. How to become productive with Git in the shortest time possible

Introduction

Git saves snapshots of your code (and everybody else's) so you can share changes.

All Git commands are *always* entered by typing `git` followed by the name of the command. For example: `git status` tells you the current status of your Git project.

If it looks like this (literally, this sentence), it's just something you read in this document.

If it looks `like this`, it's something you type on your computer at a command prompt.

The absolute minimum you need to know

There are exactly **four things** you need to do *regularly*:

Command	What It Means	When to Do It
`git clone`	Download a project from GitHub	Once, when you start working on a project
`git status`	Show you a ton of useful info about your project, including what steps you might want to take next	All the time. Whenever in doubt. Just to feel good. (Seriously, I execute `git status` before *and* after almost every single other Git command.)
`git commit`	Save a snapshot of your changes on your computer only	After you finish a small piece of work (every 30 min–1 hour)
`git push`	Upload your snapshots to GitHub	When you want other people to see your work
`git pull`	Download other people's snapshots from GitHub	Before you start a work session, or when teammates tell you they pushed

That's it. Everything else is either advanced or solving problems you haven't encountered yet.

The Git workflow

Step 1: Get the code

Look for the 

button on GitHub. Click it.

If *you are going to make changes* to the code, make sure you clone using SSH, like:

```
git clone git@github.com:KSU-AstroFlashes-SDL/FlashSat-1_Software.git
```

If you are only going to use or read the code, clone using HTTPS, like:

```
git clone https://github.com/KSU-AstroFlashes-SDL/sdl-sw-bootstrap.git
```

This downloads the entire project folder to your computer. It's a one-time thing. In the first example, you'll have a new folder named "FlashSat-1_Software" and in the second example, you'll have a new folder named "sdl-sw-bootstrap." They're both Git projects, but you'll only be able to push changes to "FlashSat-1_Software" because that's the one you cloned using SSH.

Step 2: Before you start work

```
git pull
```

This grabs any new changes your teammates pushed. **Always do this first.** It prevents conflicts later.

If you end up putting in a long session, you can (should) do this a few times. (But make sure you at least commit – if not also push – *your* changes first.)

Step 3: Make your changes

Edit your files in your normal code editor. VS Code is what we use on the Software team. You can use whatever you want, just understand that if you encounter an issue using "Super Elite Code Editor ZX Prime" because you heard it was great, you might find that no one is able to help you. So… use VS Code .

Nothing special happens when you edit your code. You can break things, delete lines, whatever. Git doesn't care until you tell it to.

Step 4: Save a snapshot (commit)

Tell Git to care about any changes you've made:

```
git add .
```

Tell Git to *store a snapshot* of your changes with a short description:

```
git commit -m "fixed the flux capacitor"
```

The message should be **short** and describe **what you did** (e.g., "fixed bug" isn't helpful). Your future self and your teammates will read this to understand the nature of your changes.

Step 5: Upload to GitHub (push)

```
git push
```

This uploads your snapshots so teammates can see and use your work. (Think about it like this: when you push, they pull; when *they* push, *you* pull. And that's how teamwork makes the dream work.)

Handling mistakes

We're humans writing arcane incantations for machines that will execute whatever instructions we provide. **Mistakes ***will*** happen.**

"I **committed** something but realized it's wrong."

If you haven't pushed yet:

```
git reset --soft HEAD~1
```

This undoes your most recent *commit* but keeps your code *changes*. Fix it and commit again.

If the mistake was three commits ago, you'd type HEAD~3 instead, just realize that you'll need to re-commit *everything* from those three commits again. Depending on how much work/change that represents, you might just consider the next option…

"I **pushed** something but realized it's wrong."

No worries. This process is no different than making a change in the first place. First, fix the code in your editor. Then (with a better message than below, obviously):

```
git add .
git commit -m "fixed the broken thing from earlier commit"
git push
```

"I pulled the latest changes, but now Git says I have conflicts."

Perfectly normal. This happens when you and a teammate both edited the same line(s) in the same file(s). Git will tell you which files have conflicts:

```
git status
```

Open those files in your editor and you'll see sections of lines marked with <<<<<<< and >>>>>>> (“conflict markers”). These show the two conflicting versions. **Pick which one is correct, or combine them**, delete the conflict markers, then:

```
git add .
git commit -m "resolved merge conflict in the blah-blah-bah function"
git push
```

The mental model (analogy)

Imagine your code is a document in Google Docs.

- **Git** keeps a complete history of every revision.
- **Commit** saves a version with a note ("Fixed typo in Chapter 3").
- **Push** makes that version visible to your teammates.
- **Pull** checks if your teammates saved new versions and, if so, downloads them.

For now, you don't need to understand *how* Git stores things internally, *why* commits use weird-looking strings of numbers and letters (they’re called “hashes”), or *what* the .git folder contains.

What comes next, after you’re comfortable (or at least productive)

- the *.gitignore* file
- branching, merging, rebasing, tagging
- `git rebase` vs. `git merge`
- `git stash`
- “detached HEAD” states